Machine Learning Classification of Structural Protein Sequences for Drug Discovery

Paul London, M.S. & Ernest Bonat, Ph.D.

December 24, 2025

Overview

Advances in drug discovery and protein engineering increasingly rely on computational models to analyze, classify, and predict protein behavior. Complex interactions between proteins and prospective drug candidates are time-consuming and expensive to manually explore. *In silico* studies are much cheaper, faster, and more efficient for narrowing down candidates that merit further experimental review.

Our work explores how modern Machine Learning (ML) techniques—particularly natural language processing (NLP), Long Short-Term Memory (LSTM), and large language models (LLM)—can be applied to amino acid sequences as if they were "biological text" using their one letter codes *(Appendix 1).* By reframing proteins as language, we can leverage powerful sequence-based models to extract features, identify patterns, and generate representations that support downstream tasks such as classification or function prediction. This work began as an attempt to build a lightweight, GPU-friendly workflow for protein analysis and model development, while maintaining scientifically meaningful accuracy.

Here we will provide a brief introduction to structural proteomics to establish some key terminology. Proteins are composed of one or more chains of amino acid residues. The specific order of these residues is known as the primary structure. Depending on this sequence and the chain's three-dimensional arrangement, local structural patterns such as folds, helices, and sheets can form, which are referred to as secondary structures. The overall three-dimensional shape of a single protein chain, incorporating these elements, is called the tertiary structure. Finally, when a protein consists of multiple chains, their combined arrangement is referred to as the quaternary structure.

Methods

Data Acquisition

Data were obtained from Kaggle (https://www.kaggle.com/datasets/shahir/protein-data-set), which were originally obtained from Research Collaboratory for Structural Bioinformatics (RCSB) Protein Data Bank (PDB) (https://www.rcsb.org/pdb/). They consist of two separate files: protein metadata and the with amino acid sequences for all chains of the protein (*Appendix 2, 3*).

The PDB archive is a repository of atomic coordinates and other information describing proteins and other important biological macromolecules. Structural biologists use methods such as X-ray crystallography, NMR spectroscopy, and cryo-electron microscopy to determine the location of each atom relative to each

other in the molecule. They then deposit this information, which is then annotated and publicly released into the archive by the PDB.

The constantly-growing PDB is a reflection of the research that is happening in laboratories worldwide. This can make it both exciting and challenging to use the database in research and education. Structures are available for many of the proteins and nucleic acids involved in the central processes of life. The PDB archive also contains structures for ribosomes, oncogenes, drug targets, and even whole viruses.

However, it can often be a challenge to find relevant information, since the PDB archives so many different structures. You will often find multiple structures for a given molecule, or partial structures, or structures that have been modified or inactivated from their native form. Therefore, significant filtering and preprocessing of the data is required.

Data Preprocessing

We utilized multiple modeling approaches to compare performance across architectures. Therefore, the data were preprocessed according to the requirements of each method. In all cases, the input variable (feature) is the amino acid sequence itself, while the output variable (target) is the functional classification of the protein.

The overall preprocessing workflow is below:

General

1. Merge both CSVs (metadata and sequences) on 'structure id' column
2. Filter to only include macromolecule type 'protein' (not DNA or RNA)
3. Drop rows with missing sequence or classification data
4. Rename columns for consistency

Target (Protein Classification)

5. Filter to only include the top 20 most represented protein classes
6. Check for uniqueness of classes with FuzzyWuzzy

Feature (Protein Sequence)

7. Add column for sequence length (for EDA)
8. Reduce sequences to just the 20 standard amino acids using 'X' as a placeholder (*Figure 1)*.
9. Filter for sequences that are 20-1024 amino acids in length
10. Add column 'combined sequence' that concatenates all chains of each protein
11. Remove duplicate 'structure id' rows created by Step 10

Modeling

12. Assign feature to *X* and target to *y*
13. Split dataset into training (70%), validation (15%), and test (15%) sets
14. Label encode *y*
15. Pipeline-specific preprocessing of *X* (see below)

```python
# Ensure sequences contain only the standard 20 amino acids
valid_aas = set('ACDEFGHIKLMNPQRSTVWY')  # standard 20 amino acids

def clean_sequence(seq):
    seq = seq.upper().strip()
    return ''.join([aa for aa in seq if aa in valid_aas])

# Maintain positions of invalid AAs using X placeholder - removing them would confuse model
def clean_sequence_keep_placeholder(seq):
    seq = seq.upper().strip()
    return ''.join([aa if aa in valid_aas else 'X' for aa in seq])

data_filtered['sequence'] = data_filtered['sequence'].apply(clean_sequence)

# We will also exclude very short sequences (< 20)
min_len = 20
data_filtered = data_filtered[data_filtered['sequence'].str.len() >= min_len].reset_index(drop=True)

# We will exclude very long sequences (> 700) at this step and match it to the tokenizer later
max_len = 1024
data_filtered = data_filtered[data_filtered['sequence'].str.len() <= max_len].reset_index(drop=True)

# We also need to combine sequences for proteins with multiple chains
df_combined_sequences = (
    data_filtered.groupby('structure_id')['sequence']
      .apply(lambda seqs: ''.join(seqs))
      .reset_index(name='combined_sequence')   # Put combined sequence into a new column
)

# Merge new column back into original data
data_filtered = data_filtered.merge(df_combined_sequences, on='structure_id', how='left')

# Remove duplicate structure IDs to account for combined sequences
data_filtered = data_filtered.drop_duplicates(subset='structure_id')
```

Figure 1. Code snippet outlining protein sequence (feature) data preprocessing steps 8 to 11.

General preprocessing included merging both CSVs on structure id, filtering to only include proteins, dropping rows missing either the target or feature, and renaming columns for consistency.

Next, target preprocessing included an assessment of how many sequences were representative of each protein classification (*Figure 2*). Thresholds of 50 classifications (green) and 200 sequences (red) were plotted to assist with selection. Ultimately, the top 20 most represented classes were utilized as the target to ensure adequate ML quality. Before modeling, the classes were label encoded.

Further, the top 20 classes were double checked for redundancy with FuzzyWuzzy, but appeared to all be distinct due to the curated nature of the dataset from RCSB. Protein classes with a slash (e.g. “HYDROLASE/HYDROLASE INHIBITOR”) can be interpreted as just hydrolase inhibitors or regulators – the dataset format treats this as a subcategory but it is still a distinct class of proteins.

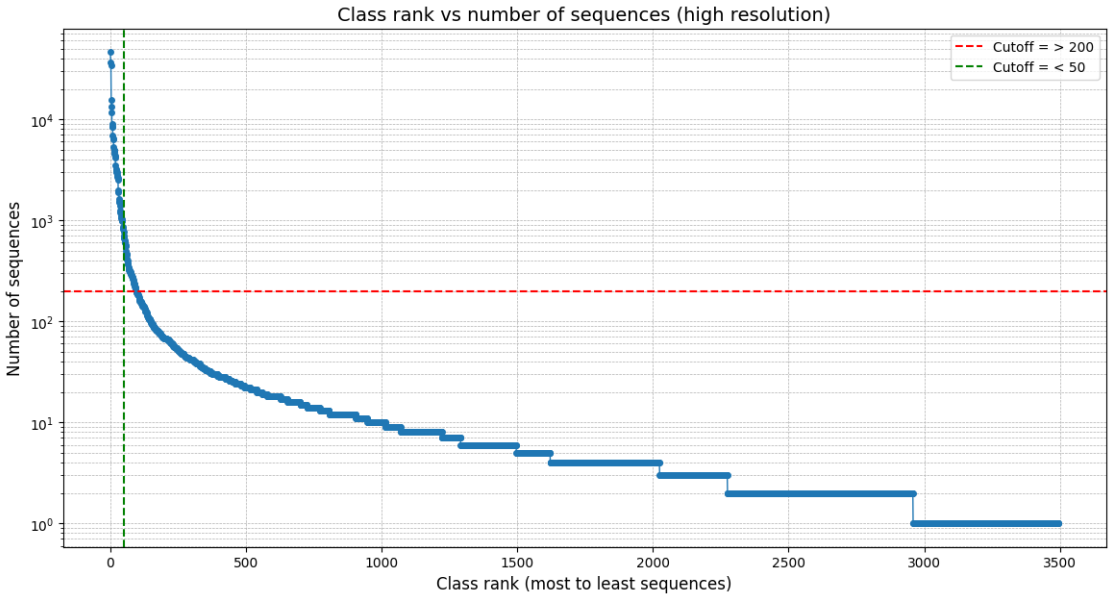


Figure 2. Class rank versus number of sequences in dataset. Thresholds: Green = 50; Red = 200

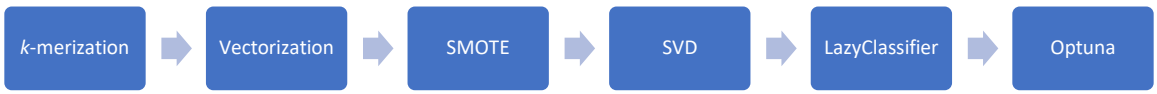
Feature preprocessing included removing ambiguous residues beyond the primary 20 amino acids, handling variable sequence lengths by setting minimum and maximum limits (20 - 1024), and combining chains for proteins with multiple strands (quaternary structure). After this, the protein sequences were preprocessed by *k*-merization and vectorization (for NLP) or tokenization (for LSTM) their amino acid characters, converting them into numeric formats suitable for ML (see next section for pipeline-specific details). This step was not required for our LLM approach, as it uses embeddings computed from a pre-trained model.

Finally, the refined data were assigned to *X* and *y* variables, randomly split into training (70%), validation (15%), and test (15%) sets, and *y* was label encoded. The data were then used for modeling.

Modeling

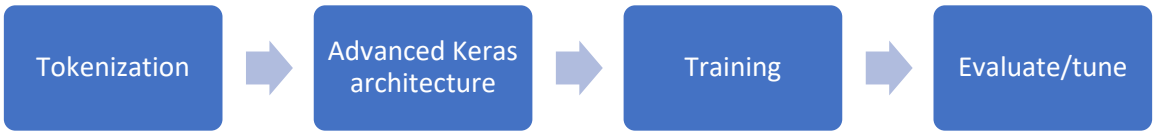
For this study, we developed three parallel pipelines:

Pipeline 1. NLP-based feature extraction using techniques commonly applied to text and tree-based models



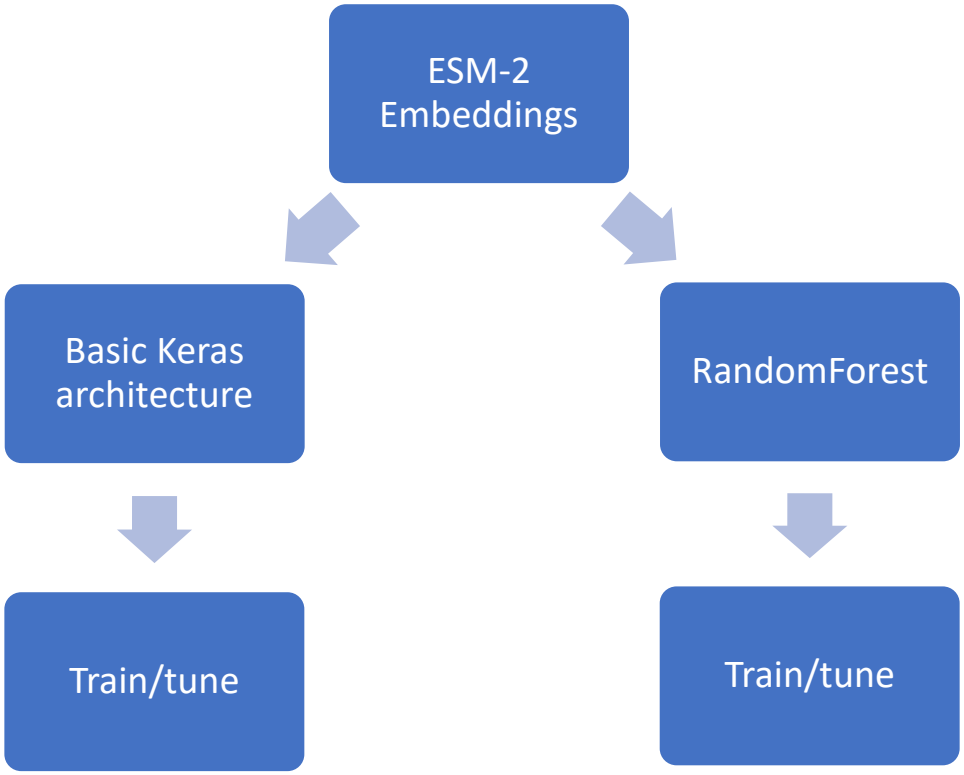
1. *k*-merization (*k* = 3)
2. Vectorization (CountVectorizer)
3. Synthetic Minority Over-sampling Technique (SMOTE)
4. Dimensionality reduction (Singular Value Decomposition (SVD))
5. LazyClassifier baseline (establish top 3 candidate models)
6. Hyperparameter tuning (Optuna) of top 3 models

Pipeline 2. LSTM-based classification trained directly on sequences padded or truncated to consistent lengths



1. Tokenization with suffix padding
2. Creation of Keras (TensorFlow) model architecture
3. Model training over 50-100 epochs
4. Review performance and tune parameters

Pipeline 3. LLM-based sequence embeddings using a pre-trained transformer model designed for biological data, enabling downstream classification with minimal training



1. Generate embeddings from ESM-2 (*esm2_t6_8M_UR50D*, Evolutionary Scale Modeling 2, Meta AI)
2. Use embeddings as input for best NLP model (Random Forest)
3. Creation of more basic Keras model architecture
4. Use embeddings as input for model training over 50-100 epochs
5. Review and compare performance and tune parameters

Each pipeline takes preprocessed sequences and outputs classification results, allowing direct performance comparison across methods. Evaluation includes accuracy score, confusion matrix, and classification report.

Exploratory Data Analysis

After data processing, we performed several analyses to explore the protein sequence data and see what could be applied to future modeling projects.

Sequence Length Distribution

After inspecting sequence lengths, we saw a left-skewed distribution, with most being 200-250 residues long and very few being greater than 1000 (*Figure 3*). Before modeling, we applied a length filter (min = 20, max = 1024) to capture the most possible signal without overloading the models.

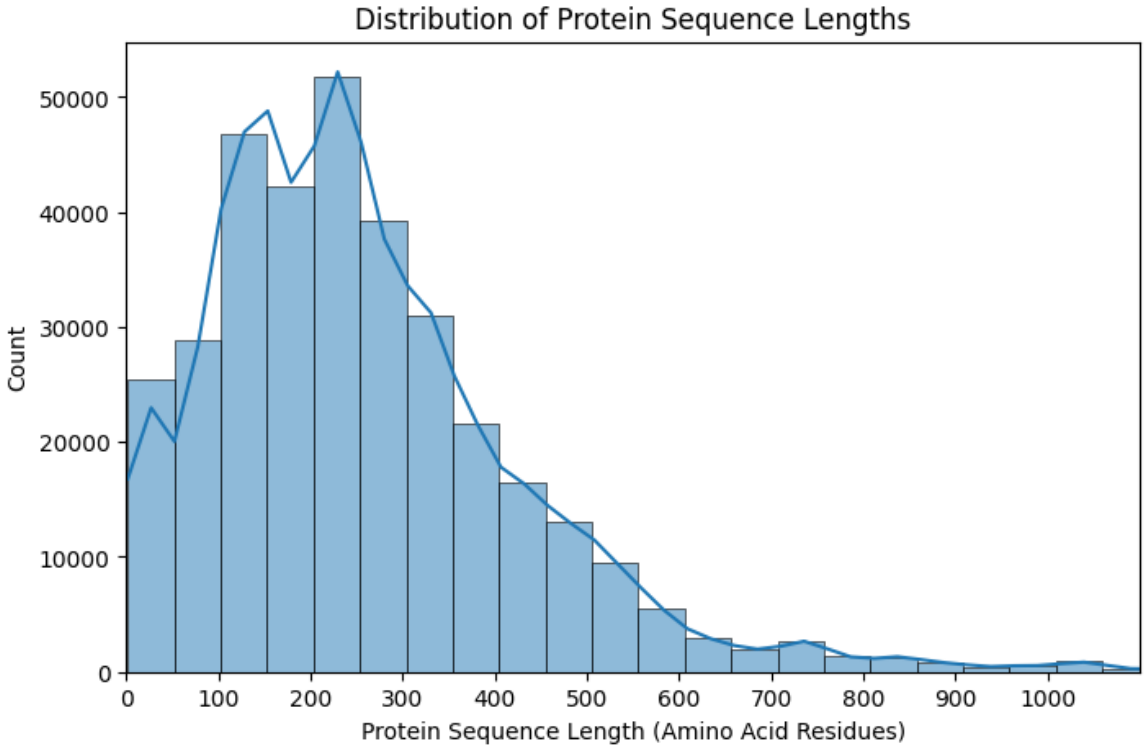


Figure 3. Distribution of protein sequence lengths.

Protein Classification Distribution

After filtering out the top 20 protein classes, we can see that the data is dominated by hydrolases, transferases, and oxidoreductases, while the others are steadily represented less (*Figure 4*). This is the primary reason SMOTE (Pipeline 1) and class weighting (Pipelines 2 and 3) were used.

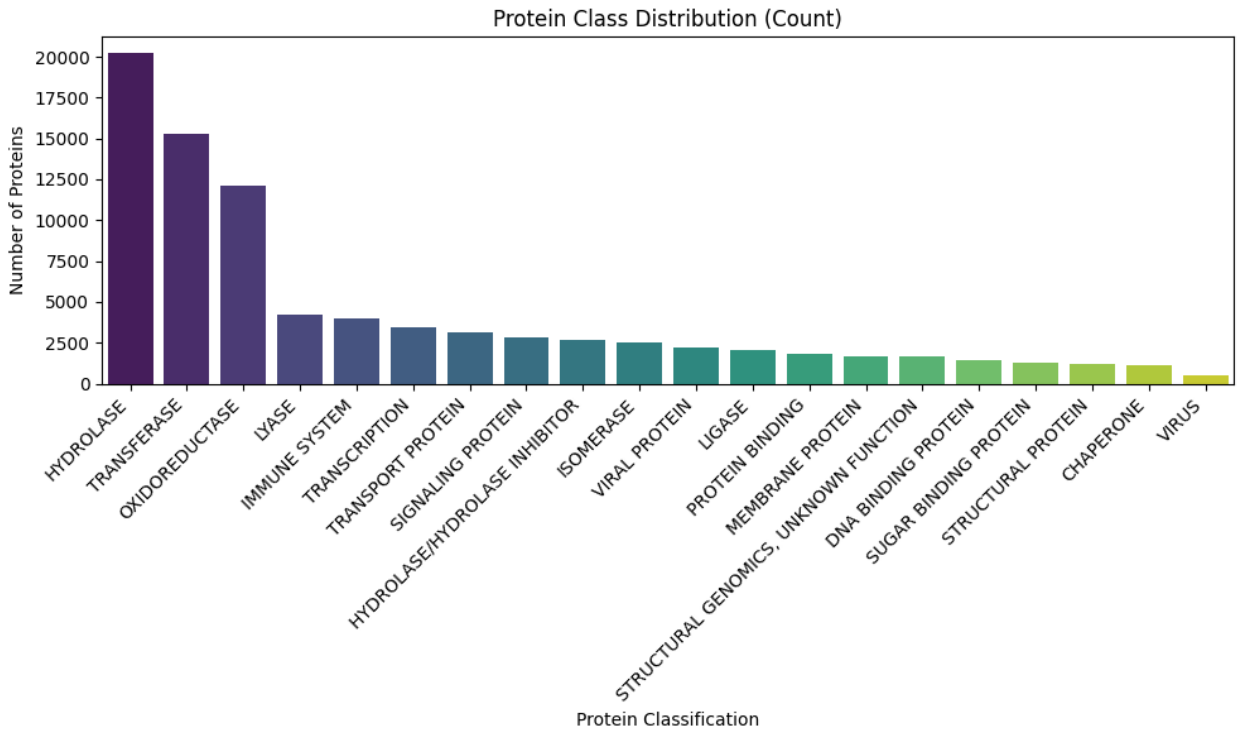


Figure 4. Distribution of protein classification.

For a summary of each of the protein classes, please see Appendix 4.

Amino Acid Composition

Beyond sequence-level analysis, which takes in to account the presence and order of amino acid residues, we can explore composition, that is, the amount of each residue present in the sequence. Further, by grouping amino acids into five distinct biochemical groups (*Appendix 5)* based on their properties, we can determine if there is any distinction between the protein classes (*Figure 5*).

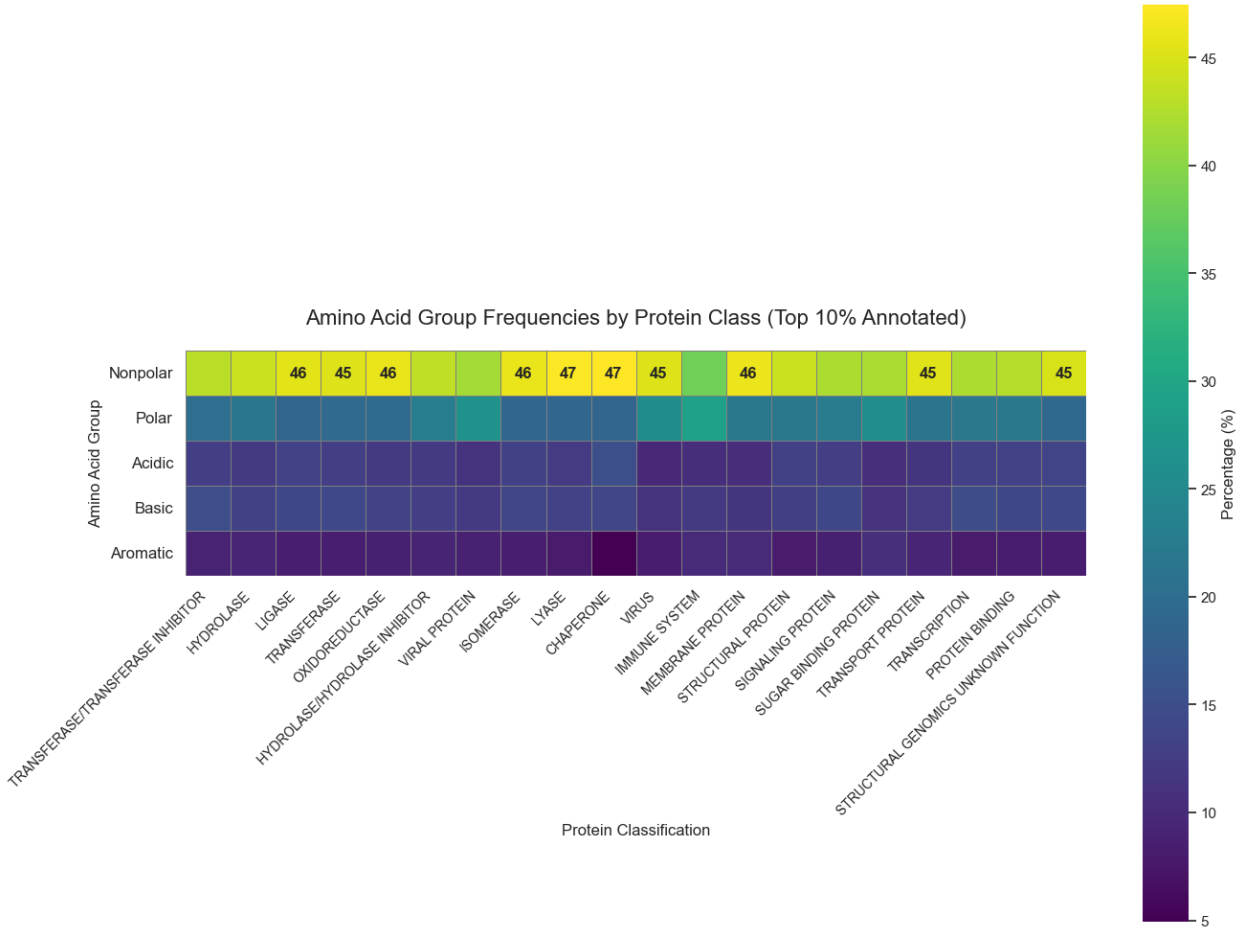


Figure 5. Amino acid biochemical group frequency by protein class.

There were not many clear patterns or distinctions from this particular plot. It appears that sequence is much more complex and essential than merely composition, since the entire “vocabulary” is only 20 amino acids, therefore they are re-used greatly.

Secondary Structure

As discussed earlier, secondary structure is an important consideration in protein function. Two major motifs seen in nearly all proteins are α-helices and β-sheets. Both stabilized by hydrogen bonds, α-helices are coiled regions of a protein, forming a spring-like structure. β-sheets are flattened, pleated regions where strands of the protein run alongside each other. These elements are vital to protein function because they form the core structural elements of proteins, providing stability, shape, and scaffolding for the overall three-dimensional structure. They also help position amino acids in the right orientation for protein function, such as binding, catalysis, or forming interaction surfaces.

For our dataset, we chose to create a Logomaker figure (*Figure 6*) of the first 30 residues of the top 5 protein classes. This illustrates patterns within these sections of the sequences: Overlay color is propensity to form helices (red) or sheets (yellow) while letter color represents the biochemical group of the amino acid: nonpolar (green), polar (blue), acidic (red), basic (purple), or aromatic (orange).

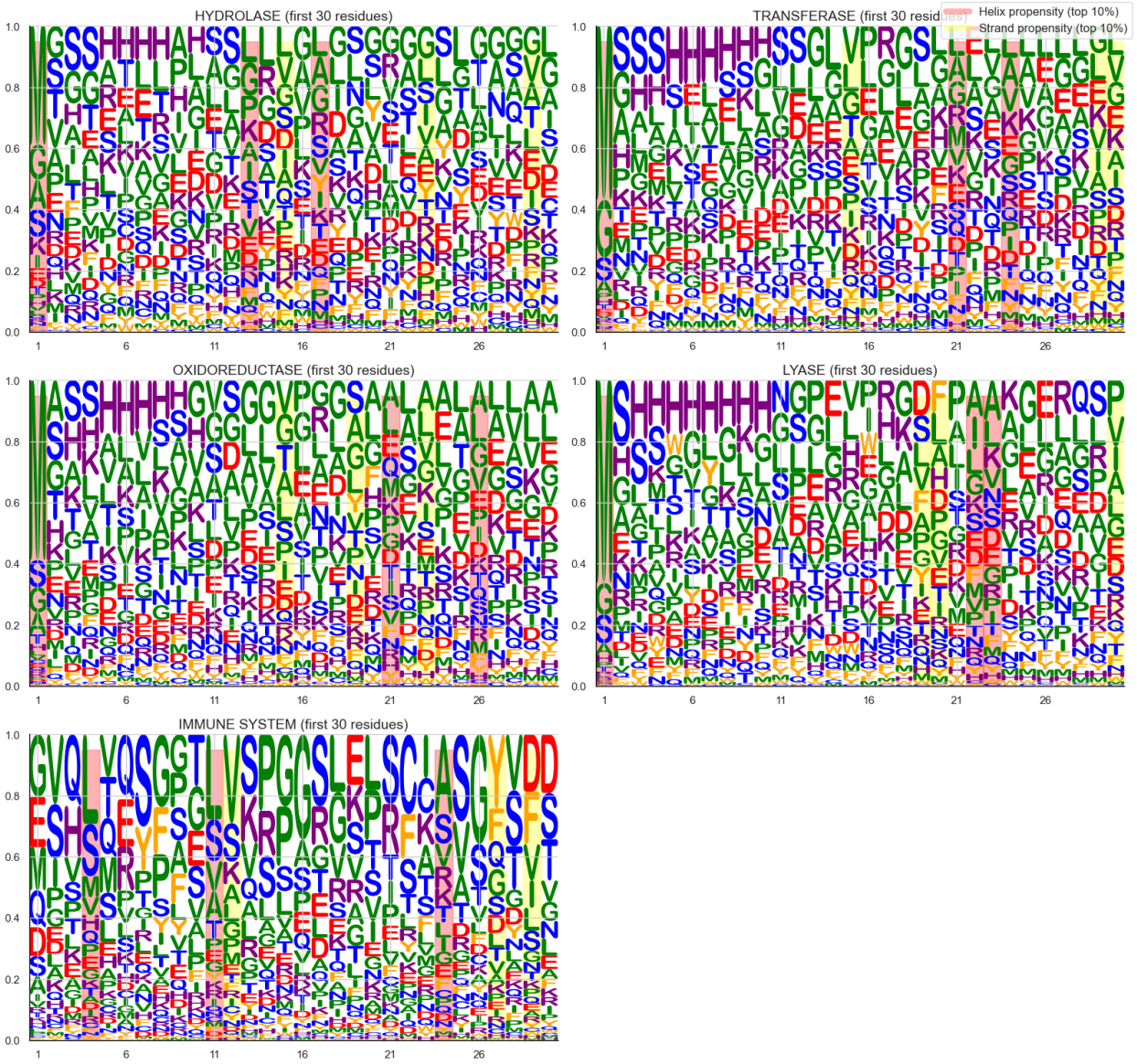


Figure 6. Frequency of the most common amino acid residues within the first 30 positions for the top five protein classes. Letter colors indicate the biochemical group of each residue: nonpolar (green), polar (blue), acidic (red), basic (purple), and aromatic (orange). Overlay shading represents predicted secondary structure propensity: α-helices (red) and β-sheets (yellow).

As shown, there are differences, but this analysis is limited to just the first 30 residues – ML models will have to take in to consideration the entire sequence, including local and distant relationships, in order to provide a better method of classification from sequence alone.

Biochemically Similar Protein Types

We also constructed a PCA plot of the top 5 protein classes grouped according to their biochemical amino acid composition (*Figure 7*).

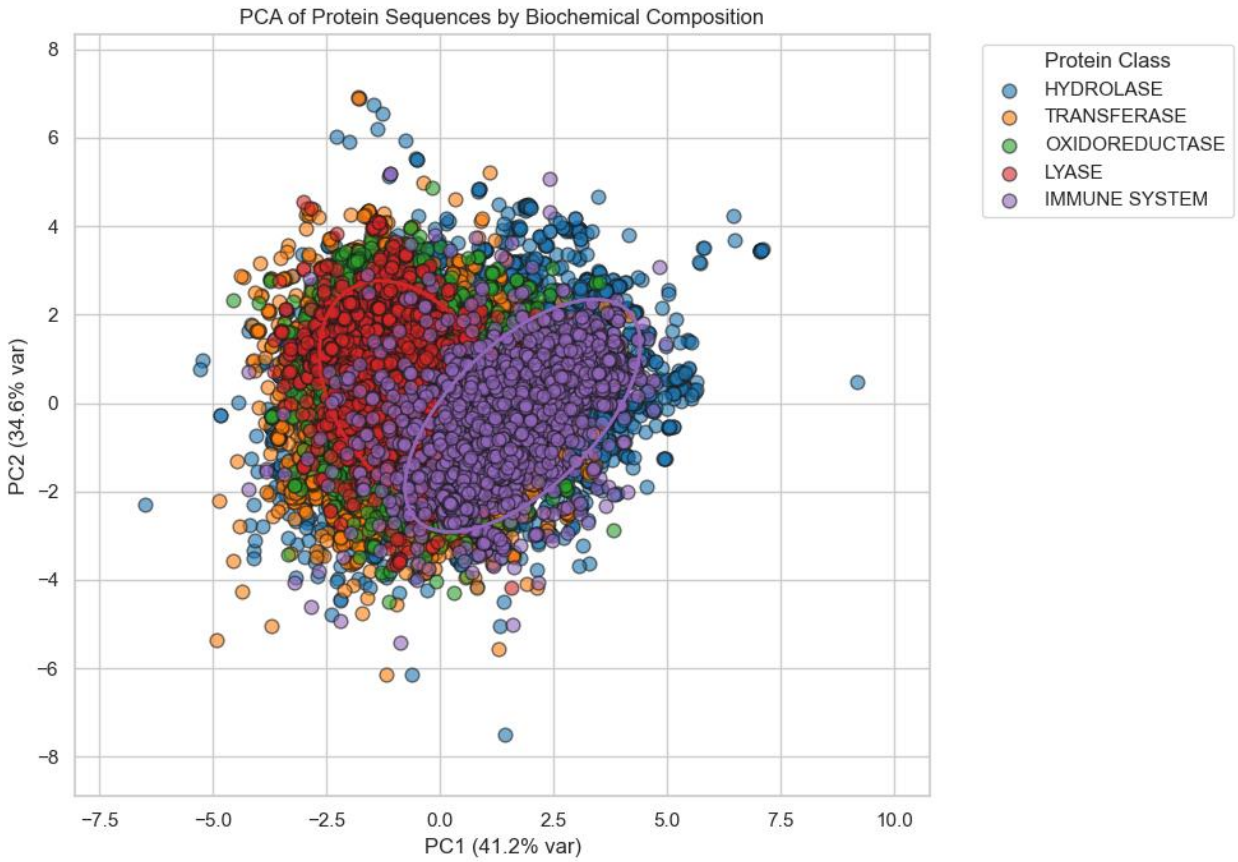


Figure 7. PCA plot of top 5 protein classes vs. biochemical amino acid composition. Each point represents a protein, and the confidence ellipses indicate 2 standard deviations (2 σ) around the mean of each class.

There is a clear distinction between lyases (red, ellipse = 2 σ) and immune system proteins (purple, ellipse = 2 σ). However, with 20 protein classes, and a great amount of overlap seen in the plots, it is clear more robust methods, such as ML, are required.

Pipeline 1: NLP

Natural language processing provides a conceptual framework for treating protein sequences as text strings composed of "tokens" (amino acids). In this project, we applied standard NLP techniques including *k*-merization, vectorization (with CountVectorizer), SMOTE, SVD, baseline exploration with LazyClassifier, and hyperparameter tuning with Optuna *(Figures 9-12)*. These approaches help capture local patterns in the sequence text, analogous to motifs in proteins.

The goal of this approach to modeling was to determine whether simple NLP architectures could extract enough signal from raw sequences to identify biologically relevant classes. While lightweight and easy to compute without a powerful GPU, these methods tend to capture only short-range dependencies. Still, they established a baseline for more advanced architectures and helped shape the direction of the project.

Among the top three tree models analyzed (Random Forest, Extra Trees, and Bagging), RandomForest performed the best with a tuned validation accuracy of 83.7% that dropped to a test accuracy of 69.7% (*Appendix 6*). This relatively low accuracy is expected for such complex data. More advanced neural networks will be required to capture the relationship between sequences and protein classification.

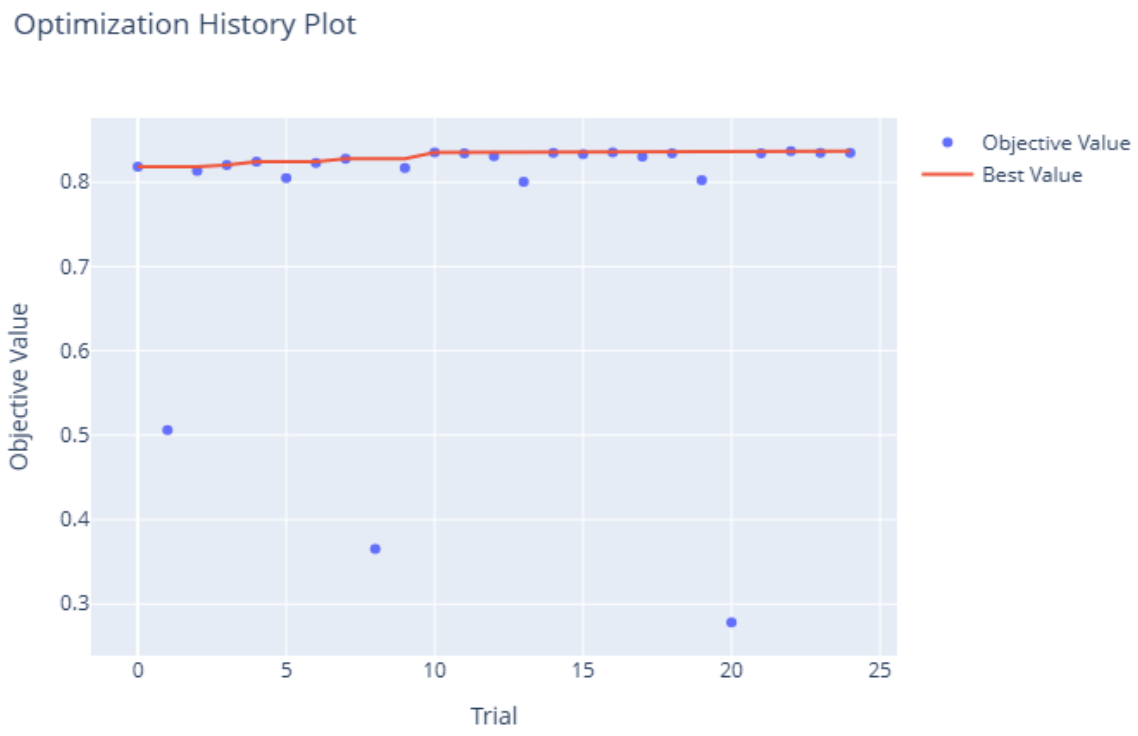


Figure 9. Optuna History Plot showing fairly stable accuracy across 25 trials.

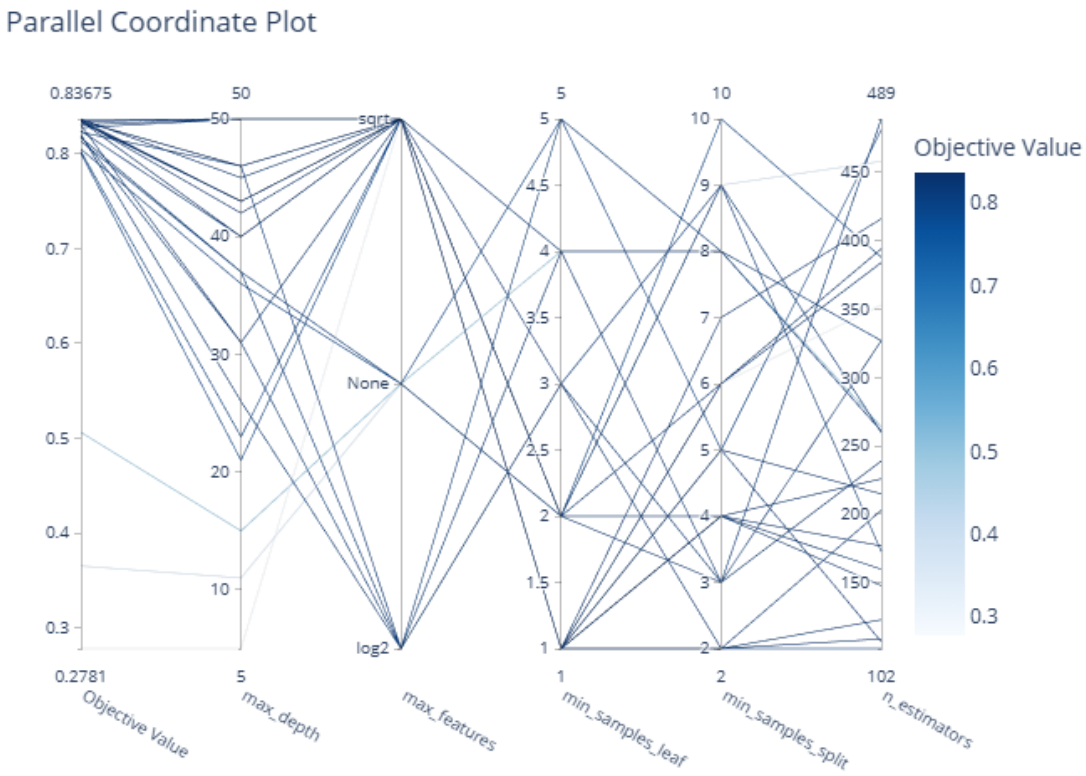


Figure 10. Optuna Parallel Coordinate Plot showing hyperparameter relationship.

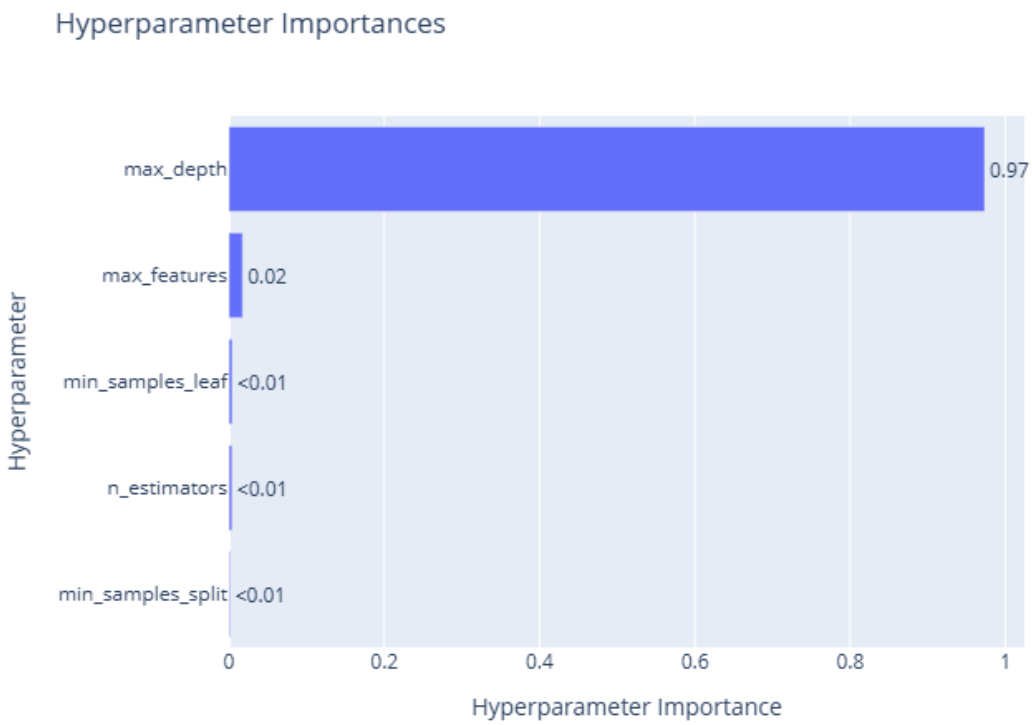


Figure 11. Optuna Hyperparameter Importances plot indicating the importance of max_depth.

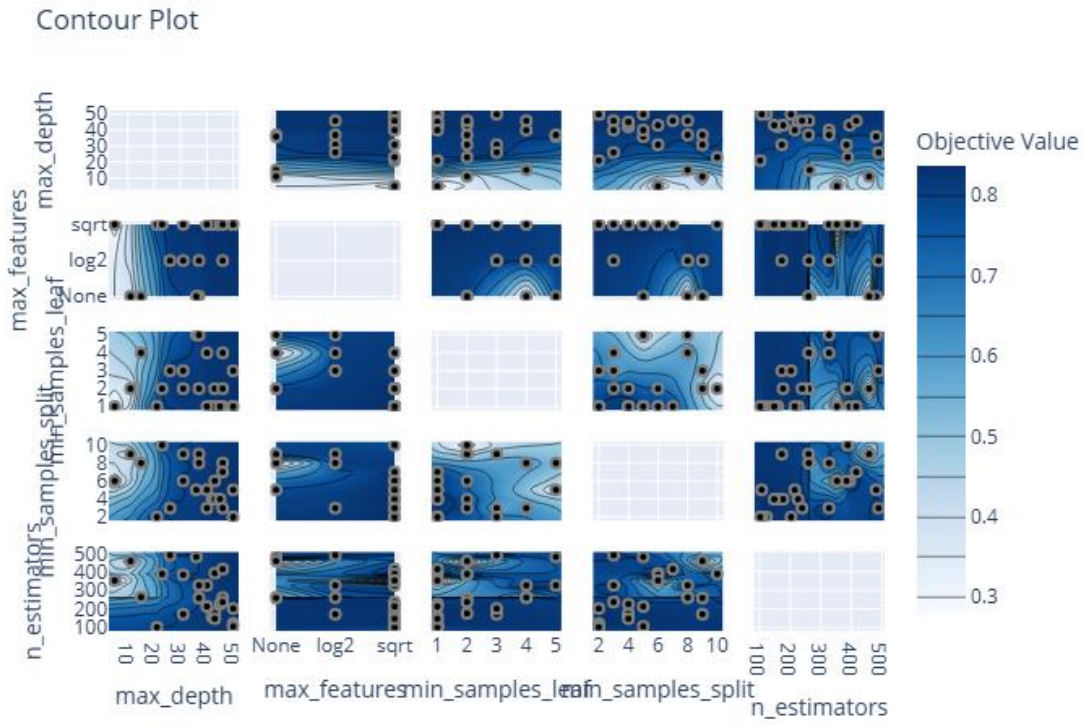


Figure 12. Optuna Contour Plot showing pairwise relationships between hyperparameters and their effect on the objective function.

Pipeline 2: LSTM

LSTMs are well-suited to sequence data, making them a natural next step. While other sequence-based architectures such as Gated Recurrent Units (GRUs), one-dimensional Convolutional Neural Networks (1D CNNs), and Transformer models could also capture sequence dependencies, LSTM networks were chosen here for their balance of expressiveness and computational efficiency on modest hardware. Unlike simple NLP models, LSTMs can capture long-range dependencies across a protein sequence, which is important because structural and functional features often depend on distant amino acid interactions.

In this project, we trained a bidirectional LSTM (BiLSTM) classifier (Keras/TensorFlow) on the cleaned, tokenized, and padded protein sequences. Despite significant hardware limitations on my local machine, the model reached a promising 78.2% accuracy on the validation set (*Figure 13, Appendix 7)*, demonstrating that traditional sequence models remain useful for biological data.

However, training was slow without GPU acceleration, and scaling up to more advanced model architecture proved difficult. For data of this complexity, multiple neural network layers are required to capture underlying signal, since much of it comes from local as well as long-range dependencies within the sequence. This reflects the biochemical reality that protein function is shaped by interactions among both proximal and distal residues along the polypeptide chain.

These limitations ultimately motivated the shift toward pre-trained embeddings (Pipeline 3) from a larger model (this approach).

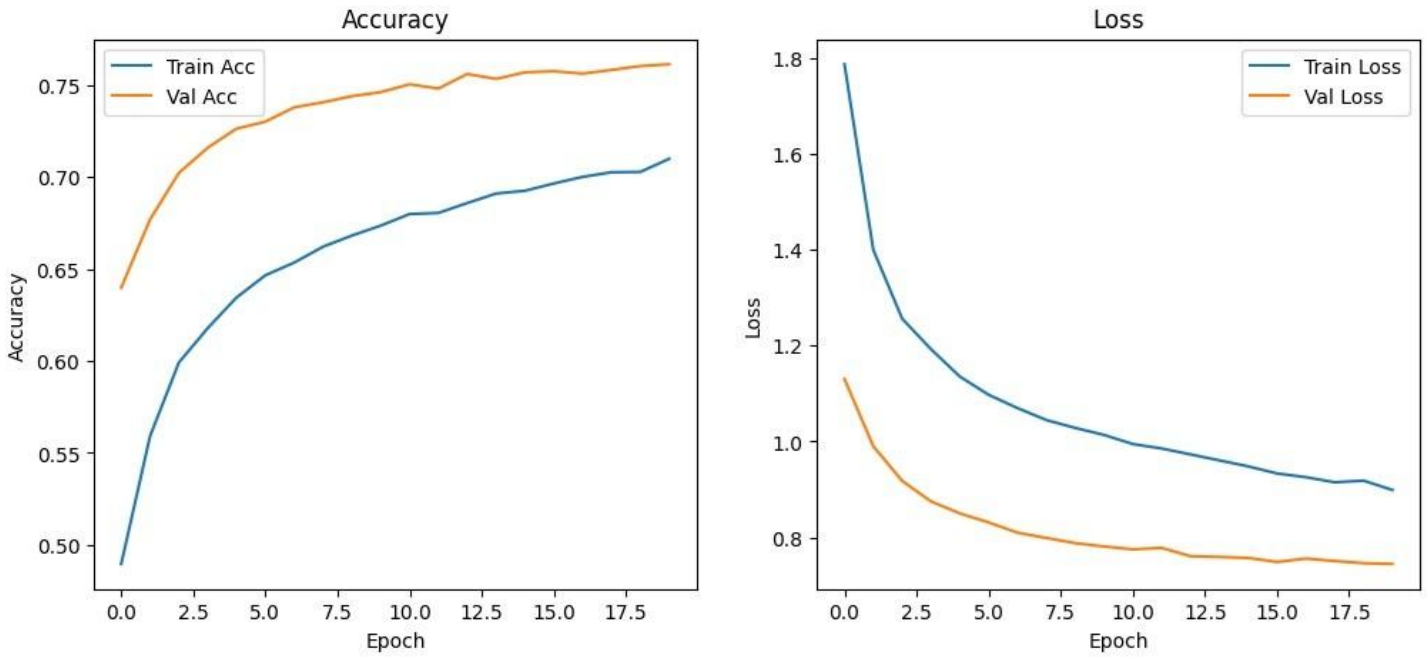


Figure 13. Training and validation accuracy and loss of Keras BiLSTM model over 20 epochs.

Pipeline 3: LLM

LLMs pre-trained on protein corpora (such as ESM, ProtBERT, etc.) offer a powerful alternative to training models from scratch. These models generate high-dimensional embeddings that capture structural, evolutionary, and biochemical relationships learned from millions of sequences. In essence, the majority of the

effort put into Pipeline 2 (LSTM) is already done by these models, reducing the required computation power exponentially.

In this approach, we used LLM-based embeddings as fixed features for downstream classification. Instead of training a deep model from scratch, sequences were passed through a pre-trained model (*Table 1; esm2_t6_8M_UR50D*, ESM-2, Meta AI) to obtain embeddings, which were then planned to be used to train two lightweight classifiers: a random forest and a shallow neural network. This strategy would have significantly improved performance and avoided the GPU limitations that slowed LSTM training.

Feature	**Value**
ESM-2 model variant	esm2_t6_8M_UR50D
Number of layers	6
Number of parameters	8 million
Training corpus	UniRef50
Embedding dimension per residue (D)	320
Output embedding shape	Sequence length (N) × 320
Effective output embedding shapes (N × D)	(20, 320) – (1024, 320)
Flattened effective output embedding lengths (N * D)	6400 – 327,680

Table 1. Properties of esm2_t6_8M_UR50D model for protein embeddings.

However, generating the ESM-2 embeddings in their entirely proved to be computationally taxing, and as such we were only able to generate 1000 total – 200 per top 5 protein class. These embeddings revealed biologically meaningful clustering patterns during exploratory analysis, visualized with a UMAP plot (Figure 14). This suggests that the model learns rich, generalizable representations of protein space. However, due to the high dimensionality of ESM-2 embeddings (sequence length × 320), it is difficult to fully capture these relationships in only two dimensions. As seen in the figure, there are distinct clusters of embeddings, but the differences between protein classes are not easily distinguished.

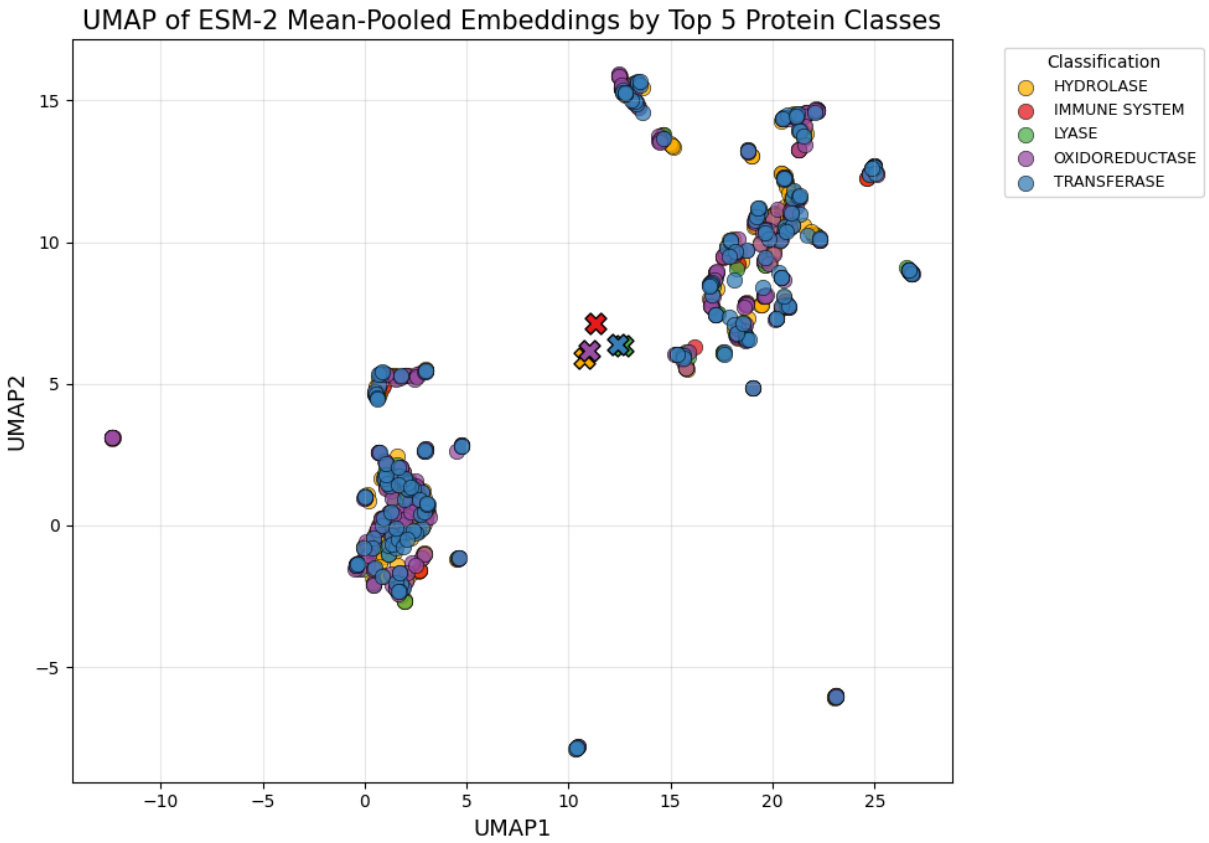


Figure 14. UMAP plot of ESM-2-generated embeddings for protein sequence data, colored by protein classification. Each point represents a sequence embedding, and there are 200 per protein class.

Once we are able to generate embeddings for the full dataset, rather than the limited 1,000 sequences used here, we expect the LLM-based approach to substantially outperform both of the other pipelines. The pre-trained model captures long-range dependencies, structural motifs, and evolutionary patterns across entire sequences, which are difficult for simpler models to learn from scratch. By leveraging the complete embedding set as input to lightweight classifiers, the model can generalize across all protein classes, handle class imbalance more effectively, and produce more accurate predictions with less training time. This scalability and rich feature representation highlight the potential of LLM embeddings as the most powerful and efficient approach in this study.

Summary of Results

Pipeline	Method	Test Accuracy (%)	Notes
1	NLP + Random Forest	69.7	Lightweight baseline using 3-mers & SVD
2	Bidirectional LSTM	78.2	Best sequence-based model so far
3	ESM-2 LLM Embeddings	In progress	Strongest clustering; evaluation ongoing

Discussion

This project highlights how different ML methods interact with protein sequence data. Traditional NLP methods offer simplicity and speed but struggle to capture complex dependencies. LSTMs improve model expressiveness but are constrained by computational resources when trained from scratch on large datasets.

By contrast, LLM-based approaches provide high-quality embeddings with minimal effort, suggesting that modern pre-trained models are the most efficient path forward for many protein analysis tasks—especially when hardware is limited.

Further, future modeling should expand the featureset to other fields, such as pH, rather than protein sequence alone. This approach would increase the robustness of ML models dramatically.

Although the project remains a work in progress, the results demonstrate the advantages of applying language-based models to biological sequences and underscore the growing convergence between computational biology and modern AI. Future steps include expanding the dataset, fine-tuning transformer models, and exploring generative AI capabilities for protein analysis and drug design. This so-called “Intelligent Drug Design” appears to be the new way forward to solve the intricate biochemical interactions required by the compounds to have the desired therapeutic effects.

Appendix

One-Letter Code	Amino Acid
A	Alanine
R	Arginine
N	Asparagine
D	Aspartic Acid
C	Cysteine
E	Glutamic Acid
Q	Glutamine
G	Glycine
H	Histidine
I	Isoleucine
L	Leucine
K	Lysine
M	Methionine
F	Phenylalanine
P	Proline
S	Serine
T	Threonine
W	Tryptophan
Y	Tyrosine
V	Valine

Appendix 1. One Letter Amino Acid Code

Metadata – *pdb_data_no_dups.csv* – (141401 rows × 14 columns)

Column	Name/Description
structureId	Unique identifier for each protein structure in the PDB (Protein Data Bank)
classification	Structural class or category of the protein (e.g., enzyme, transporter)
experimentalTechnique	Method used to determine the protein structure (e.g., X-ray crystallography, NMR, cryo-EM)
macromoleculeType	Type of macromolecule (e.g., protein, DNA, RNA)
residueCount	Number of amino acid residues in the protein chain
resolution	Resolution of the protein structure (Å), relevant for X-ray crystallography
structureMolecularWeight	Molecular weight of the protein structure (Daltons)
crystallizationMethod	Method used for crystallizing the protein (if applicable)
crystallizationTemp	Temperature used for protein crystallization (Kelvin or Celsius)
densityMatthews	Matthews coefficient, a measure of crystal packing density
densityPercentSol	Estimated solvent content (%) in the crystal
pdbxDetails	Additional details about the structure (text description)
phValue	pH at which the protein structure was determined
publicationYear	Year the protein structure was published in the PDB

Appendix 2. Column names and descriptions for metadata portion of dataset.

Sequences – *pdb_data_seq.csv* – (467304 rows × 5 columns)

Column	Name/Description
structureId	Unique identifier for the protein structure (matches pdb_data_no_dups.csv).
chainId	Identifier for the protein chain within the structure (A, B, C, etc.).
sequence	Amino acid sequence of the chain (one-letter codes).
residueCount	Number of residues in this chain.
macromoleculeType	Type of macromolecule (e.g., protein, DNA, RNA).

Appendix 3. Column names and descriptions for sequence portion of dataset.

Category	Summary
Hydrolase	Enzymes that catalyze bond hydrolysis using water; key in metabolism and biomolecule breakdown
Transferase	Enzymes that transfer functional groups between molecules; essential in metabolism and signaling
Oxidoreductase	Catalyze redox reactions and electron transfer; central to energy production and detoxification
Immune System	Proteins involved in pathogen detection, immune signaling, antigen presentation, and defense
Hydrolase / Hydrolase Inhibitor	Hydrolase inhibitors/regulators that control proteolysis, inflammation, and tissue remodeling
Lyase	Enzymes that cleave or form chemical bonds without hydrolysis or oxidation; active in metabolic pathways
Transcription	Proteins regulating gene expression, including transcription factors and RNA polymerase components
Viral Protein	Virus-encoded proteins supporting infection, replication, immune evasion, and virion assembly
Transport Protein	Facilitate movement of ions, metabolites, and molecules across membranes or within the cell
Virus	Proteins associated with viral structure, replication, regulation, or host manipulation
Signaling Protein	Mediate and propagate cellular signals; includes kinases, phosphatases, and receptors
Isomerase	Catalyze structural rearrangements within molecules; important in metabolism and stereochemistry
Structural Genomics – Unknown Function	Experimentally solved protein structures lacking functional annotation
Ligase	Join molecules via new bonds, often ATP-dependent; crucial in DNA repair and biosynthesis
Membrane Protein	Embedded in or associated with membranes; include channels, receptors, and transporters
Protein Binding	Proteins primarily defined by binding interactions that scaffold complexes or regulate pathways
Transferase / Transferase Inhibitor	Transferase inhibitors/regulators that regulate metabolic and signaling pathways.

Structural Protein	Provide mechanical support or shape; include cytoskeletal and extracellular matrix components.
Chaperone	Assist in protein folding, stabilization, and prevention of aggregation.
Sugar Binding Protein	Bind carbohydrates for signaling, transport, or pathogen recognition (e.g., lectins).

Appendix 4. Protein classification summary.

Category	One-letter Code	Amino Acid	Notes
Nonpolar Aliphatic	A	Alanine	Small, hydrophobic
	G	Glycine	Smallest residue, flexible
	I	Isoleucine	Hydrophobic, aliphatic
	L	Leucine	Hydrophobic, aliphatic
	M	Methionine	Contains sulfur
	P	Proline	Cyclic, rigid structure
	V	Valine	Hydrophobic, aliphatic
Aromatic	F	Phenylalanine	Nonpolar, aromatic
	W	Tryptophan	Aromatic, slightly polar
	Y	Tyrosine	Polar, aromatic
Polar Uncharged	C	Cysteine	Can form disulfide bonds
	N	Asparagine	Polar, uncharged
	Q	Glutamine	Polar, uncharged
	S	Serine	Polar, uncharged
	T	Threonine	Polar, uncharged
Acidic (Negative)	D	Aspartic Acid	Acidic, negatively charged
	E	Glutamic Acid	Acidic, negatively charged
Basic (Positive)	K	Lysine	Basic, positively charged
	R	Arginine	Basic, positively charged
	H	Histidine	Basic, partially charged at physiological pH

Appendix 5. Amino acids sorted in to categories by biochemical properties

Parameter	Value	Notes
n_estimators	226	Number of trees in the forest
criterion	gini	Function to measure split quality
max_depth	45	Maximum depth of each tree
max_features	sqrt	Number of features considered at each split
min_samples_split	4	Minimum samples required to split a node
min_samples_leaf	1	Minimum samples required at a leaf node
class_weight	None	Weighting applied to classes
random_state	42	Seed for reproducibility
bootstrap	True	Whether bootstrap samples are used

Appendix 6. Hyperparameters of best Random Forest model in Pipeline 1.

Name	Class	Units/Filters	Activation
input_layer_7	InputLayer	-	-
embedding_9	Embedding	64	-
conv1d_1	Conv1D	128	relu
batch_normalization_3	BatchNormalization	-	-
max_pooling1d_1	MaxPooling1D	[3]	-
bidirectional_6	Bidirectional	-	-
batch_normalization_4	BatchNormalization	-	-
dense_14	Dense	128	relu
batch_normalization_5	BatchNormalization	-	-
dropout_8	Dropout	rate=0.4	-
dense_15	Dense	20	softmax

Appendix 7. Parameters of best Keras model in Pipeline 2.